Bandit LAB - script for the complete presentation

16 slides, about 8 min 40 s of speech (plus the demo). Speak slowly, pause at full stops, and look up at the TA between sentences.

Speaker 1 - slides 1-5: problem, MDP, RL maths, DQN

Slide 1 (~20 s)

Good afternoon, everyone. We're Bandit LAB. Today we'll present our implementation of “Social Attention for Autonomous Decision-Making in Dense Traffic”, by Leurent and Mercat from INRIA and Renault, published at a NeurIPS 2019 workshop. We reproduced the paper, and then we extended it in two ways.

Slide 2 (~30 s)

Picture a car that has to turn left at a busy junction with no traffic light. Go too early, and it crashes. Wait too long, and it never gets through. That's called the freezing robot problem. Real drivers handle this by watching only the few cars that matter. Writing rules for every case is impossible, so the car learns by trial and error, with reinforcement learning.

Slide 3 (~40 s)

We describe the task as a Markov Decision Process. The state is a table with one row per car: our car plus up to fourteen others, and seven numbers per row: whether the car is there, its position, its velocity and its heading. We scale everything to between minus one and one. Every second, the agent picks slower, idle or faster; the steering is automatic. The reward at the bottom gives up to one point per second for driving fast, minus five for a crash, and one for reaching the exit.

Slide 4 (~35 s)

The agent wants to maximise the return: the sum of future rewards, where rewards further away count a bit less, because of gamma, zero point nine five. The Q-value tells us how good an action is in a given state. The best Q-values satisfy the Bellman equation: the value now equals the reward now, plus gamma times the best value in the next state. And once we know Q, driving is easy: just pick the action with the highest Q.

Slide 5 (~45 s)

Since the state is continuous, a neural network estimates Q. That's DQN. We train it towards the target y, the reward plus the discounted value of the next state, by minimising the squared error. Three tricks keep training stable: a replay buffer of fifteen thousand past moments, sampled at random; a target network, a frozen copy updated every five hundred and twelve steps; and Double DQN, where one network picks the next action and the other one scores it. Exploration follows this epsilon schedule, from one down to five percent. Over to my teammate for the paper's main idea.

Speaker 2 - slides 6-10: the paper's idea, networks, attention, our implementation

Slide 6 (~30 s)

Thanks. The paper keeps DQN fixed and asks one question: how should the network read the other cars? A plain list changes size with the number of cars, and it depends on their order. But re-ordering the cars shouldn't change the decision. That's called permutation invariance, written here as an equation. To test it, the order of the cars is shuffled at every step, in the paper and in our code.

Slide 7 (~30 s)

The paper compares three networks of about the same size, around thirty thousand weights. FCN flattens the table into one long vector, so it isn't order-invariant. CNN uses a thirty-two by thirty-two grid seen from above;

it is order-invariant, but coarse. And Ego-Attention is the paper's model. Our weight counts match the paper's Table 1 very closely.

Slide 8 (~45 s)

Here's how ego-attention works. Each car is encoded by a small shared network into a vector e. Our car creates a query, and every car creates a key and a value. The score of each car is the query dot its key, divided by the square root of the head size. Softmax turns the scores into weights alpha that add up to one, and empty rows are masked out. The output is the weighted sum of the values, added back to our car's own encoding, and a decoder gives the three Q-values. There are two heads, so this happens twice, in parallel.

Slide 9 (~35 s)

A quick example. Scores of two, zero point five, zero and minus one point five become weights of seventy-two, sixteen, ten and two percent. So the network focuses on the dangerous car. Because it's a weighted sum, shuffling the cars changes nothing, and it works for any number of cars. And we can draw these weights as lines, like in this picture, to see what the agent is watching.

Slide 10 (~30 s)

We implemented all of this from scratch in PyTorch, and took the hyper-parameters straight from the authors' own code. We trained three agents with three seeds each, four thousand episodes per run, and tested each one on the same hundred unseen scenarios, with no exploration. Now my teammate will take you through the results.

Speaker 3 - slides 11-16: results, extensions, demo, conclusion

Slide 11 (~40 s)

Thanks. Our training curves show the same three behaviours as the paper. FCN is fast but reckless, so it crashes. CNN is too cautious and slow. Ego-

attention gets the highest reward. On the test, attention succeeds sixty-seven percent of the time with just twelve percent crashes. FCN crashes forty-one percent of the time, and CNN runs out of time in a third of the cases. With heavier traffic than in training, the ranking stays the same.

Slide 12 (~35 s)

Next, we went beyond the paper. The paper trains for one left turn, and suggests giving the agent its destination as future work. We did that: the car can start on any of four roads and go left, straight or right, twelve routes in total. We add the direction to each car's destination: for our car it's the goal, for the others it works like a turn signal. One attention agent learned all twelve routes and halved the crashes compared with FCN, with the biggest gain on left turns.

Slide 13 (~40 s)

Our second extension is a long journey through a city of six junctions, to a finish line. A shortest-path planner chooses the turns, and our agent decides when to go. At each junction we give it positions relative to that junction, so its skill transfers. Without extra training, attention finished fifty-two and a half percent of the journeys, against thirty-four for FCN. Long journeys are harder because the risks multiply, as this formula shows. And honestly, fine-tuning on the city made it worse, because it learned to be more aggressive.

Slide 14 (~25 s)

Here's our demo. The green band is the route, the lines show which cars the agent is watching, and the panel on the right shows its Q-values and its decision, second by second.

Slide 15 (~30 s)

To conclude: how the network reads the traffic matters as much as the learning algorithm. Attention is safer, works with any number of cars, and is

easy to interpret. Our limitations: it's a simulator, the other drivers follow simple rules, the car only controls its speed, and we used three seeds instead of a hundred and twenty. Next steps would be lane changes, more seeds and real traffic data.

Slide 16 (~10 s)

Thank you very much. We're happy to take your questions.